CORRECTION EXAMEN

Algorithmique et Structure de Données

TEK-UP – ING-3 – Année 2024-2025

Date : 23/01/2025 – Durée : 1H30

Barème : 5 + 5 + 10 = 20 points

Table des matières

1 Exercice 1 : Tri par Poids Numérique (5 points) 2

1.1 Partie 1 : Fonction poids_numerique (2 points) 2

1.2 Partie 2 : Procédure trier_par_poids (3 points) 3

1.3 Partie 3 : Solution avec Tableau d'Enregistrements (Alternative) 6

2 Exercice 2 : Recherche Ternaire (5 points) 9

3 Problème : Gestion de Flotte de Drones (10 points) 12

3.1 Question 1 : Définition des structures de données (1.5 point) 12

3.2 Question 2 : Ajouter un drone (1.5 point) 14

3.3 Question 3 : Ajouter un colis (1.5 point) 16

3.4 Question 4 : Effectuer une livraison (1.5 point) 18

3.5 Question 5 : Chargement des drones (2 points) 20

3.6 Question 6 : Planification des livraisons (2 points) 22

4 Récapitulatif des Points Clés 24

1 Exercice 1 : Tri par Poids Numérique (5 points)

Énoncé

Écrire un algorithme qui permet de trier les éléments d'un tableau d'entiers en fonction de leurs **poids numériques**, dans un ordre croissant. Le poids numérique d'un nombre est défini comme la **somme des carrés de ses chiffres**. En cas d'égalité du poids numérique, trier les nombres par ordre croissant de leurs valeurs.

Exemple : Pour [15, 81, 23, 7, 45]

- 15 : $1^2 + 5^2 = 1 + 25 = 26$
- 81 : $8^2 + 1^2 = 64 + 1 = 65$
- 23 : $2^2 + 3^2 = 4 + 9 = 13$
- 7 : $7^2 = 49$
- 45 : $4^2 + 5^2 = 16 + 25 = 41$

Résultat : [23, 15, 45, 7, 81] (triés par poids : 13, 26, 41, 49, 65)

1.1 Partie 1 : Fonction poids_numerique (2 points)

Analyse du problème

On doit extraire chaque chiffre d'un nombre et calculer la somme de leurs carrés.

Comment extraire les chiffres ?

- `n mod 10` donne le dernier chiffre (ex : 123 mod 10 = 3)
- `n div 10` enlève le dernier chiffre (ex : 123 div 10 = 12)

Algorithme :

1. Initialiser la somme à 0
2. Tant que le nombre n'est pas 0 :
   - Extraire le dernier chiffre
   - Ajouter son carré à la somme
   - Enlever le dernier chiffre
3. Retourner la somme

CORRECTION – Fonction poids_numerique

```
fonction poids_numerique(d n : entier) : entier
var somme, chiffre, nombre : entier
debut
    somme <-- 0
    nombre <-- n

    // Gérer les nombres négatifs (prendre la valeur absolue)
    si nombre < 0
    alors nombre <-- -nombre
    finsi

    // Cas particulier : si n = 0, le poids est 0
    si nombre = 0
    alors retourner 0
    finsi

    // Extraire chaque chiffre et calculer la somme des carrés
    tantque nombre > 0 faire
        chiffre <-- nombre mod 10       // Dernier chiffre
        somme <-- somme + chiffre * chiffre   // Ajouter son carré
        nombre <-- nombre div 10        // Enlever le dernier chiffre
    fintantque

    retourner somme
fin
```

Trace d'exécution pour n

Itération	nombre	chiffre	somme
Début	123	–	0
1	12	3	$0 + 3^2 = 9$
2	1	2	$9 + 2^2 = 13$
3	0	1	$13 + 1^2 = 14$

Résultat : poids_numerique(123) = 14

1.2 Partie 2 : Procédure trier_par_poids (3 points)

Analyse du problème

On doit trier un tableau selon deux critères :

1. **Critère principal :** le poids numérique (croissant)
2. **Critère secondaire :** la valeur du nombre (croissant, en cas d'égalité de poids)

Stratégie : Utiliser un tri classique (tri à bulles ou tri par sélection) mais modifier la condition de comparaison pour prendre en compte les deux critères.

Condition d'échange :

— Si poids(T[j]) > poids(T[j+1]) → échanger

— Si poids(T[j]) = poids(T[j+1]) ET T[j] > T[j+1] → échanger

CORRECTION – Procédure trier_par_poids (Tri à bulles)

```
procedure trier_par_poids(d/r T : tableau[1..max] de entier, d n : entier)
var i, j, temp : entier
    poids_j, poids_j1 : entier
    echange : booleen
debut
    repeter
        echange <-- faux

        pour j de 1 a n - 1 faire
            // Calculer les poids des deux éléments adjacents
            poids_j <-- poids_numerique(T[j])
            poids_j1 <-- poids_numerique(T[j + 1])

            // Condition d'échange avec les deux critères
            si (poids_j > poids_j1) ou
               ((poids_j = poids_j1) et (T[j] > T[j + 1]))
            alors
                // Échanger T[j] et T[j + 1]
                temp <-- T[j]
                T[j] <-- T[j + 1]
                T[j + 1] <-- temp
                echange <-- vrai
            finsi
        finpour

    jusqua non echange
fin
```

Alternative : Tri par sélection

```
procedure trier_par_poids_selection(d/r T : tableau, d n : entier)
var i, j, indMin, temp : entier
    poids_min, poids_j : entier
debut
    pour i de 1 a n - 1 faire
        indMin <-- i
        poids_min <-- poids_numerique(T[i])

        pour j de i + 1 a n faire
            poids_j <-- poids_numerique(T[j])

            si (poids_j < poids_min) ou
               ((poids_j = poids_min) et (T[j] < T[indMin]))
            alors
                indMin <-- j
                poids_min <-- poids_j
            finsi
        finpour

        // Échanger T[i] et T[indMin]
        si indMin <> i
        alors
            temp <-- T[i]
            T[i] <-- T[indMin]
            T[indMin] <-- temp
        finsi
    finpour
fin
```

Trace d'exécution pour [15

] **Poids calculés :**

— 15 → 26, 81 → 65, 23 → 13, 7 → 49, 45 → 41

Évolution du tri :

15, 81, 23, 7, 45 → 23 (poids 13) est le plus petit → [23, 81, 15, 7, 45]

23, 81, 15, 7, 45 → 15 (poids 26) vient ensuite → [23, 15, 81, 7, 45]

1. Continue... → **Résultat final : [23, 15, 45, 7, 81]**

1.3 Partie 3 : Solution avec Tableau d'Enregistrements (Alternative)

Idée de la solution

Principe : Créer un tableau auxiliaire où chaque case contient un **enregistrement** avec :

— La **valeur** originale du nombre

— Son **poids** numérique (pré-calculé)

Avantage : On calcule le poids UNE SEULE FOIS par élément, puis on trie selon le champ "poids". C'est plus efficace que de recalculer le poids à chaque comparaison !

Étapes :

1. Définir un type enregistrement (valeur, poids)
2. Remplir un tableau d'enregistrements avec les valeurs et leurs poids
3. Trier ce tableau selon le champ "poids" (et "valeur" en cas d'égalité)
4. Recopier les valeurs triées dans le tableau original

Déclaration du type enregistrement

```
constante max = 100

type ELEMENT_POIDS = enregistrement
                         valeur : entier    // La valeur originale
                         poids : entier     // Le poids numérique calculé
                     fin

type TAB_POIDS = tableau[1..max] de ELEMENT_POIDS
```

CORRECTION – Tri avec tableau d'enregistrements (Deux boucles imbriquées)

```
procedure trier_par_poids_enreg(d/r T : tableau[1..max] de entier, d n : entier)
var TabPoids : TAB_POIDS
    i, j : entier
    temp : ELEMENT_POIDS
debut
    // Étape 1 : Remplir le tableau d'enregistrements
    pour i de 1 a n faire
        TabPoids[i].valeur <-- T[i]
        TabPoids[i].poids <-- poids_numerique(T[i])
    finpour

    // Étape 2 : Tri croissant selon le poids (deux boucles imbriquées)
    pour i de 1 a n - 1 faire
        pour j de i + 1 a n faire
            // Comparer poids de i avec poids de j
            si TabPoids[i].poids > TabPoids[j].poids
            alors
                // Échanger les enregistrements
                temp <-- TabPoids[i]
                TabPoids[i] <-- TabPoids[j]
                TabPoids[j] <-- temp
            finsi
        finpour
    finpour

    // Étape 3 : Recopier les valeurs triées dans T
    pour i de 1 a n faire
        T[i] <-- TabPoids[i].valeur
    finpour
fin
```

CORRECTION – Tri avec tableau d'enregistrements (Tri par sélection)

```
procedure trier_par_poids_enreg_selection(d/r T : tableau[1..max] de entier,
                                          d n : entier)
var TabPoids : TAB_POIDS
    i, j, indMin : entier
    temp : ELEMENT_POIDS
debut
    // Étape 1 : Remplir le tableau d'enregistrements
    pour i de 1 a n faire
        TabPoids[i].valeur <-- T[i]
        TabPoids[i].poids <-- poids_numerique(T[i])
    finpour

    // Étape 2 : Tri croissant selon le poids (tri par sélection)
    pour i de 1 a n - 1 faire
        indMin <-- i

        pour j de i + 1 a n faire
            // Chercher le minimum selon le poids
            si TabPoids[j].poids < TabPoids[indMin].poids
            alors
                indMin <-- j
            finsi
        finpour

        // Échanger les enregistrements
        si indMin <> i
        alors
            temp <-- TabPoids[i]
            TabPoids[i] <-- TabPoids[indMin]
            TabPoids[indMin] <-- temp
        finsi
    finpour

    // Étape 3 : Recopier les valeurs triées dans T
    pour i de 1 a n faire
        T[i] <-- TabPoids[i].valeur
    finpour
fin
```

Trace d'exécution avec enregistrements

Tableau initial : T = [15, 81, 23, 7, 45]
Étape 1 – Création du tableau d'enregistrements :

Indice	valeur	poids
1	15	26
2	81	65
3	23	13
4	7	49
5	45	41

Étape 2 – Après tri selon le champ "poids" :

Indice	valeur	poids
1	23	13
2	15	26
3	45	41
4	7	49
5	81	65

Étape 3 – Tableau final : T = [23, 15, 45, 7, 81] ✓

Avantages de cette approche

1. **Efficacité :** Le poids est calculé une seule fois par élément (complexité $O(n)$), au lieu de le recalculer à chaque comparaison.
2. **Clarté :** Le tri se fait simplement sur un champ de l'enregistrement.
3. **Extensibilité :** On peut facilement ajouter d'autres champs (ex : indice original).

Inconvénient : Utilisation de mémoire supplémentaire pour le tableau d'enregistrements.

2 Exercice 2 : Recherche Ternaire (5 points)

Énoncé

La **recherche ternaire** est une extension de la recherche dichotomique qui divise l'intervalle en **trois parties** au lieu de deux.
Principe :

1. Calculer deux pivots qui divisent le tableau en 3 parties
2. Si l'élément recherché est égal à l'un des pivots → trouvé
3. Si l'élément est inférieur au premier pivot → chercher dans la 1ère partie
4. Si l'élément est supérieur au deuxième pivot → chercher dans la 3ème partie
5. Sinon → chercher dans la 2ème partie (entre les pivots)

Écrire une fonction **récursive** qui retourne la position de l'élément ou -1 s'il n'existe pas.

Comprendre la recherche ternaire

Différence avec la recherche binaire :

- Binaire : 1 pivot au milieu, 2 parties
- Ternaire : 2 pivots, 3 parties

Calcul des pivots :

- `pivot1 = debut + (fin - debut) / 3`
- `pivot2 = fin - (fin - debut) / 3`

```
                [--partie1--][--partie2--][--partie3--]
            debut        pivot1        pivot2        fin
```

Cas de base : Si debut > fin, l'élément n'existe pas → retourner -1

CORRECTION – Fonction recherche_ternaire

```
fonction recherche_ternaire(d T : tableau[1..max] de entier,
                            d elem : entier,
                            d debut, fin : entier) : entier
var pivot1, pivot2 : entier
debut
    // Cas de base : intervalle invalide
    si debut > fin
    alors retourner -1
    finsi

    // Calcul des deux pivots
    pivot1 <-- debut + (fin - debut) div 3
    pivot2 <-- fin - (fin - debut) div 3

    // Vérifier si l'élément est à l'un des pivots
    si T[pivot1] = elem
    alors retourner pivot1
    finsi

    si T[pivot2] = elem
    alors retourner pivot2
    finsi

    // Déterminer dans quelle partie chercher
    si elem < T[pivot1]
    alors
        // Chercher dans la première partie (gauche de pivot1)
        retourner recherche_ternaire(T, elem, debut, pivot1 - 1)
    sinon si elem > T[pivot2]
    alors
        // Chercher dans la troisième partie (droite de pivot2)
        retourner recherche_ternaire(T, elem, pivot2 + 1, fin)
    sinon
        // Chercher dans la deuxième partie (entre pivot1 et pivot2)
        retourner recherche_ternaire(T, elem, pivot1 + 1, pivot2 - 1)
    finsi
fin
```

Appel initial de la fonction

```
// Pour chercher un élément dans un tableau T de n éléments :
position <-- recherche_ternaire(T, element_cherche, 1, n)

si position = -1
alors ecrire("Élément non trouvé")
sinon ecrire("Élément trouvé à la position ", position)
finsi
```

Trace d'exécution

Tableau : T = [2, 5, 8, 12, 16, 23, 38, 56, 72, 91] (indices 1 à 10)
Recherche de : 23

Appel	debut	fin	pivot1	pivot2	T[p1], T[p2]	Action
1	1	10	4	7	12, 38	$12 < 23 < 38 \rightarrow$ partie 2
2	5	6	5	6	16, 23	T[6] = 23 $\rightarrow$ **trouvé !**

Résultat : Position 6

Complexité de la recherche ternaire

- **Recherche binaire :** $O(\log_2 n)$
- **Recherche ternaire :** $O(\log_3 n)$

Attention : Bien que $\log_3 n < \log_2 n$, la recherche ternaire fait **2 comparaisons** par itération contre 1 pour la binaire. En pratique, la binaire est souvent plus efficace !

3 Problème : Gestion de Flotte de Drones (10 points)

Contexte du problème

Une entreprise de logistique utilise des **drones** pour livrer des **colis** à domicile.
Caractéristiques d'un colis :

- Identifiant unique (entier)
- Poids (en kg)
- Priorité (haute, normale, basse)
- Adresse de destination (latitude et longitude GPS)

Caractéristiques d'un drone :

- Identifiant unique (entier)
- Capacité de chargement maximale (en kg)
- Niveau de batterie (en pourcentage)
- Emplacement actuel (latitude et longitude GPS)
- Statut (disponible, en mission, en charge)
- Liste des colis transportés (liste chaînée)

3.1 Question 1 : Définition des structures de données (1.5 point)

Analyse des besoins

On a besoin de :

1. Un **enregistrement pour les coordonnées GPS** (latitude, longitude)
2. Un **enregistrement pour les colis** + une liste chaînée de colis
3. Un **enregistrement pour les drones** + une liste chaînée de drones

Types énumérés à définir :

- Priorité : {haute, normale, basse}
- Statut : {disponible, en_mission, en_charge}

CORRECTION – Définition des types

```
// Types énumérés
type PRIORITE = (haute, normale, basse)
type STATUT = (disponible, en_mission, en_charge)

// Structure pour les coordonnées GPS
type ADRESSE = enregistrement
                   latitude : reel
                   longitude : reel
               fin

// Structure pour un colis (noeud de la liste chaînée de colis)
type PTR_COLIS = ^COLIS
type COLIS = enregistrement
                 id : entier
                 poids : reel
                 priorite : PRIORITE
                 destination : ADRESSE
                 suiv : PTR_COLIS
             fin

// Structure pour un drone (noeud de la liste chaînée de drones)
type PTR_DRONE = ^DRONE
type DRONE = enregistrement
                 id : entier
                 capacite_max : reel
                 batterie : entier          // Pourcentage (0-100)
                 position : ADRESSE
                 statut : STATUT
                 colis : PTR_COLIS          // Tête de la liste des colis
                 suiv : PTR_DRONE
             fin

// La flotte est une liste chaînée de drones
var flotte : PTR_DRONE     // Tête de la liste des drones
```

Points importants à retenir

- Chaque **drone** contient un pointeur vers une **liste de colis**
- La **flotte** est elle-même une liste chaînée de drones
- C'est une structure imbriquée : liste de listes !

3.2 Question 2 : Ajouter un drone (1.5 point)

Analyse

Deux cas possibles :

1. Le drone **n'existe pas** → l'ajouter à la flotte (liste de colis vide)
2. Le drone **existe déjà** (même id) → mettre à jour ses informations

Stratégie :

1. Chercher si un drone avec le même id existe
2. Si oui : mettre à jour ses champs
3. Si non : créer un nouveau noeud et l'ajouter en tête

CORRECTION – Procédure ajouter_drone

```
procedure ajouter_drone(d/r flotte : PTR_DRONE,
                        d id : entier,
                        d capacite : reel,
                        d batterie : entier,
                        d pos : ADRESSE,
                        d stat : STATUT)
var p, nouveau : PTR_DRONE
    trouve : booleen
debut
    // Étape 1 : Chercher si le drone existe déjà
    p <-- flotte
    trouve <-- faux

    tantque (p <> nil) et (non trouve) faire
        si p^.id = id
        alors trouve <-- vrai
        sinon p <-- p^.suiv
        finsi
    fintantque

    si trouve
    alors
        // Cas 1 : Le drone existe, mettre à jour ses informations
        p^.capacite_max <-- capacite
        p^.batterie <-- batterie
        p^.position <-- pos
        p^.statut <-- stat
        // Note : on ne modifie PAS la liste des colis existante
    sinon
        // Cas 2 : Le drone n'existe pas, le créer et l'ajouter
        allouer(nouveau)
        nouveau^.id <-- id
        nouveau^.capacite_max <-- capacite
        nouveau^.batterie <-- batterie
        nouveau^.position <-- pos
        nouveau^.statut <-- stat
        nouveau^.colis <-- nil      // Liste de colis initialement vide
        nouveau^.suiv <-- flotte    // Insertion en tête
        flotte <-- nouveau
    finsi
fin
```

3.3 Question 3 : Ajouter un colis (1.5 point)

Analyse

On doit :

1. Trouver le drone correspondant à l'id donné
2. Dans la liste de colis de ce drone :
   - Si le colis existe (même id) → le mettre à jour
   - Sinon → l'ajouter

CORRECTION – Procédure ajouter_colis

```
procedure ajouter_colis(d flotte : PTR_DRONE,
                        d id_drone : entier,
                        d id_colis : entier,
                        d poids_colis : reel,
                        d prio : PRIORITE,
                        d dest : ADRESSE)
var p_drone : PTR_DRONE
    p_colis, nouveau : PTR_COLIS
    drone_trouve, colis_trouve : booleen
debut
    // Étape 1 : Trouver le drone
    p_drone <-- flotte
    drone_trouve <-- faux

    tantque (p_drone <> nil) et (non drone_trouve) faire
        si p_drone^.id = id_drone
        alors drone_trouve <-- vrai
        sinon p_drone <-- p_drone^.suiv
        finsi
    fintantque

    // Vérifier que le drone existe
    si non drone_trouve
    alors retourner      // Ou afficher une erreur
    finsi

    // Étape 2 : Chercher si le colis existe dans la liste du drone
    p_colis <-- p_drone^.colis
    colis_trouve <-- faux

    tantque (p_colis <> nil) et (non colis_trouve) faire
        si p_colis^.id = id_colis
        alors colis_trouve <-- vrai
        sinon p_colis <-- p_colis^.suiv
        finsi
    fintantque

    si colis_trouve
    alors
        // Cas 1 : Le colis existe, mettre à jour
        p_colis^.poids <-- poids_colis
        p_colis^.priorite <-- prio
        p_colis^.destination <-- dest
    sinon
        // Cas 2 : Le colis n'existe pas, l'ajouter en tête
        allouer(nouveau)
        nouveau^.id <-- id_colis
        nouveau^.poids <-- poids_colis
        nouveau^.priorite <-- prio
        nouveau^.destination <-- dest
        nouveau^.suiv <-- p_drone^.colis
        p_drone^.colis <-- nouveau
    finsi
fin
```

3.4 Question 4 : Effectuer une livraison (1.5 point)

Analyse

Supprimer un colis = supprimer un noeud dans une liste chaînée.
Rappel : Pour supprimer un noeud, on doit :

1. Trouver le noeud à supprimer
2. Mettre à jour le pointeur du noeud précédent
3. Libérer la mémoire du noeud supprimé

Cas particulier : Si c'est le premier colis de la liste, il faut modifier `p_drone^.colis`

CORRECTION – Procédure effectuer_livraison

```
procedure effectuer_livraison(d flotte : PTR_DRONE,
                              d id_drone : entier,
                              d id_colis : entier)
var p_drone : PTR_DRONE
    p_colis, prec : PTR_COLIS
    drone_trouve, colis_trouve : booleen
debut
    // Étape 1 : Trouver le drone
    p_drone <-- flotte
    drone_trouve <-- faux

    tantque (p_drone <> nil) et (non drone_trouve) faire
        si p_drone^.id = id_drone
        alors drone_trouve <-- vrai
        sinon p_drone <-- p_drone^.suiv
        finsi
    fintantque

    si non drone_trouve
    alors retourner
    finsi

    // Étape 2 : Trouver et supprimer le colis
    p_colis <-- p_drone^.colis
    prec <-- nil
    colis_trouve <-- faux

    tantque (p_colis <> nil) et (non colis_trouve) faire
        si p_colis^.id = id_colis
        alors colis_trouve <-- vrai
        sinon
            prec <-- p_colis
            p_colis <-- p_colis^.suiv
        finsi
    fintantque

    si colis_trouve
    alors
        // Supprimer le colis de la liste
        si prec = nil
        alors
            // C'est le premier colis
            p_drone^.colis <-- p_colis^.suiv
        sinon
            // C'est un colis au milieu ou à la fin
            prec^.suiv <-- p_colis^.suiv
        finsi

        liberer(p_colis)
    finsi
fin
```

3.5 Question 5 : Chargement des drones (2 points)

Analyse

On cherche le(s) drone(s) avec :

1. La plus grande **capacité disponible** (capacité max - poids des colis actuels)
2. Un niveau de batterie > 50%

Algorithme :

1. Pour chaque drone avec batterie > 50% :
   - Calculer le poids total des colis transportés
   - Calculer la capacité disponible = capacité max - poids total
2. Retourner la liste des drones ayant la capacité disponible maximale

Note : On retourne une liste car plusieurs drones peuvent avoir la même capacité maximale.

CORRECTION – Fonction auxiliaire : calculer_poids_total

```
fonction calculer_poids_total(d liste_colis : PTR_COLIS) : reel
var p : PTR_COLIS
    total : reel
debut
    total <-- 0
    p <-- liste_colis

    tantque p <> nil faire
        total <-- total + p^.poids
        p <-- p^.suiv
    fintantque

    retourner total
fin
```

CORRECTION – Fonction identifier_drones_disponibles

```
fonction identifier_drones_disponibles(d flotte : PTR_DRONE) : PTR_DRONE
var p : PTR_DRONE
    resultat, nouveau, dernier : PTR_DRONE
    cap_dispo, max_cap : reel
debut
    // Première passe : trouver la capacité maximale
    max_cap <-- -1
    p <-- flotte

    tantque p <> nil faire
        si p^.batterie > 50
        alors
            cap_dispo <-- p^.capacite_max - calculer_poids_total(p^.colis)
            si cap_dispo > max_cap
            alors max_cap <-- cap_dispo
            finsi
        finsi
        p <-- p^.suiv
    fintantque

    // Deuxième passe : collecter tous les drones avec cette capacité
    resultat <-- nil
    dernier <-- nil
    p <-- flotte

    tantque p <> nil faire
        si p^.batterie > 50
        alors
            cap_dispo <-- p^.capacite_max - calculer_poids_total(p^.colis)
            si cap_dispo = max_cap
            alors
                // Ajouter ce drone à la liste résultat
                allouer(nouveau)
                nouveau^.id <-- p^.id
                nouveau^.capacite_max <-- p^.capacite_max
                nouveau^.batterie <-- p^.batterie
                nouveau^.position <-- p^.position
                nouveau^.statut <-- p^.statut
                nouveau^.colis <-- p^.colis
                nouveau^.suiv <-- nil

                si resultat = nil
                alors
                    resultat <-- nouveau
                    dernier <-- nouveau
                sinon
                    dernier^.suiv <-- nouveau
                    dernier <-- nouveau
                finsi
            finsi
        finsi
        p <-- p^.suiv
    fintantque
```



```
    retourner resultat
fin
```

Version simplifiée : Retourner UN SEUL drone

Si on demande seulement LE drone (pas les drones), voici une version plus simple :

```
fonction drone_meilleure_capacite(d flotte : PTR_DRONE) : PTR_DRONE
var p, meilleur : PTR_DRONE
    cap_dispo, max_cap : reel
debut
    meilleur <-- nil
    max_cap <-- -1
    p <-- flotte

    tantque p <> nil faire
        si p^.batterie > 50
        alors
            cap_dispo <-- p^.capacite_max - calculer_poids_total(p^.colis)
            si cap_dispo > max_cap
            alors
                max_cap <-- cap_dispo
                meilleur <-- p
            finsi
        finsi
        p <-- p^.suiv
    fintantque

    retourner meilleur
fin
```

3.6 Question 6 : Planification des livraisons (2 points)

Analyse

On doit trier la liste des colis d'un drone selon :

1. **Critère principal :** la priorité (haute < normale < basse)
2. **Critère secondaire :** la distance entre le drone et la destination

Fonctions données :

- `distance(adr_destination, adr_drone)` : calcule la distance GPS
- `permuter_colis(c1, c2)` : échange les informations de deux colis

Comparaison des priorités :

- haute = 0, normale = 1, basse = 2 (ordre naturel de l'énumération)
- haute < normale < basse en valeur numérique

CORRECTION – Procédure planifier_livraisons (Tri à bulles)

```
procedure planifier_livraisons(d flotte : PTR_DRONE, d id_drone : entier)
var p_drone : PTR_DRONE
    p, q : PTR_COLIS
    echange : booleen
    dist_p, dist_q : reel
debut
    // Étape 1 : Trouver le drone
    p_drone <-- flotte

    tantque (p_drone <> nil) et (p_drone^.id <> id_drone) faire
        p_drone <-- p_drone^.suiv
    fintantque

    si p_drone = nil
    alors retourner      // Drone non trouvé
    finsi

    // Étape 2 : Trier la liste des colis (tri à bulles sur liste chaînée)
    repeter
        echange <-- faux
        p <-- p_drone^.colis

        tantque (p <> nil) et (p^.suiv <> nil) faire
            q <-- p^.suiv

            // Calculer les distances
            dist_p <-- distance(p^.destination, p_drone^.position)
            dist_q <-- distance(q^.destination, p_drone^.position)

            // Condition d'échange :
            // - Si priorité de p > priorité de q (p moins prioritaire)
            // - OU si mêmes priorités et distance de p > distance de q
            si (p^.priorite > q^.priorite) ou
               ((p^.priorite = q^.priorite) et (dist_p > dist_q))
            alors
                permuter_colis(p, q)
                echange <-- vrai
            finsi

            p <-- p^.suiv
        fintantque

    jusqua non echange
fin
```

Pourquoi utiliser permuter_colis ?

Dans une liste chaînée, il est plus simple d'échanger les **contenus** des noeuds plutôt que de modifier les pointeurs. La procédure `permuter_colis` échange :

— `id`, `poids`, `priorite`, `destination`

Elle NE touche PAS au pointeur `suiv` !
Alternative : On aurait pu manipuler les pointeurs, mais c'est plus complexe et source d'erreurs.

Ordre de priorité

Dans l'énumération `type PRIORITE = (haute, normale, basse)` :

— `haute` vaut 0

— `normale` vaut 1

— `basse` vaut 2

Donc `haute < normale < basse` en termes de valeur.
Quand on trie par priorité **croissante**, les colis **haute** priorité viennent en premier !

4 Récapitulatif des Points Clés

Ce qu'il faut retenir de cet examen

Exercice 1 – Tri par poids :

— Extraction de chiffres : `mod 10` et `div 10`

— Tri avec critère composé (deux conditions)

Exercice 2 – Recherche ternaire :

— Division en 3 parties avec 2 pivots

— Bien gérer les 4 cas (trouvé aux pivots, ou 3 intervalles)

— Cas de base : `debut > fin`

Problème – Drones :

— Listes chaînées imbriquées (liste de drones, chacun avec liste de colis)

— Mise à jour vs insertion (chercher d'abord si existe)

— Suppression dans liste chaînée (garder le précédent)

— Tri sur liste chaînée (permuter les contenus, pas les pointeurs)

Erreurs classiques à éviter

1. **Oublier d'initialiser** les pointeurs à `nil`
2. **Ne pas gérer le cas** où la liste est vide
3. **Perdre le pointeur** vers un élément à supprimer avant de faire `liberer`
4. **Oublier le cas particulier** de suppression en tête de liste
5. **Confondre** `p` (le pointeur) et `p^` (le contenu)

Bonne révision !

Rappel : Toujours vérifier les cas limites (liste vide, élément non trouvé, etc.)